

## *Memory Efficient Doubly Linked List*

### *–Insert At Position and Time Complexity*

#### *Program:*

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    // Traverse to find the node BEFORE the insertion point
    (pos-1)
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    while (curr != nullptr && i < pos - 1)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```

```

    }

    // We are inserting AFTER 'curr'.
    // Let 'next' be the node currently after 'curr'.
    next = XOR(prev, curr->npx);

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Linkage Logic:
    // 1. Setup newNode: npx = curr ^ next
    newNode->npx = XOR(curr, next);

    // 2. Update curr: npx was (prev ^ next), now becomes
    (prev ^ newNode)
    curr->npx = XOR(prev, newNode);

    // 3. Update next (if it exists): npx was (curr ^
    nextNext), now becomes (newNode ^ nextNext)
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted at position " << pos << "."
    << endl;
}

...
case 3:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter position: ";
    cin >> pos;
    insertAtPosition(data, pos);
    break;

```

### **Program Analysis:**

```
if (pos < 1)
{
    cout << "Invalid position!" << endl;
    return;
}
```

*if pos is less than 1 , it will print ``Invalid position!``  
and return void and exit from the function.*

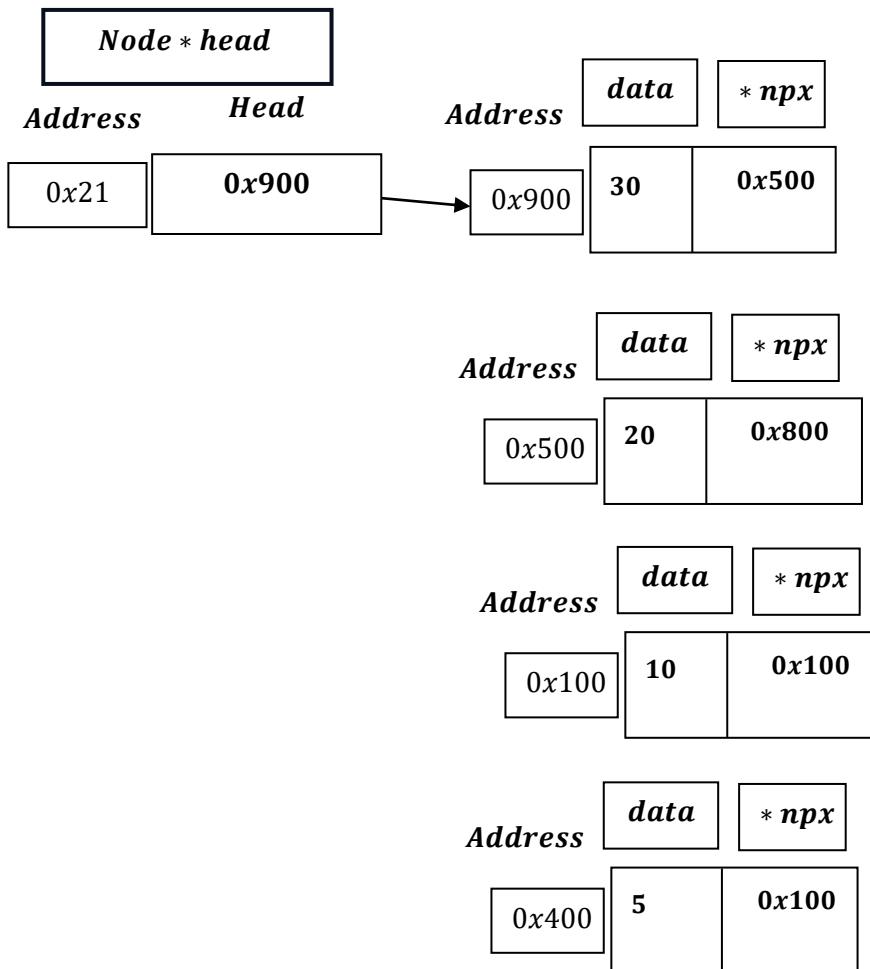
---

```
if (pos == 1)
{
    insertAtBeginning(data);
    return;
}
```

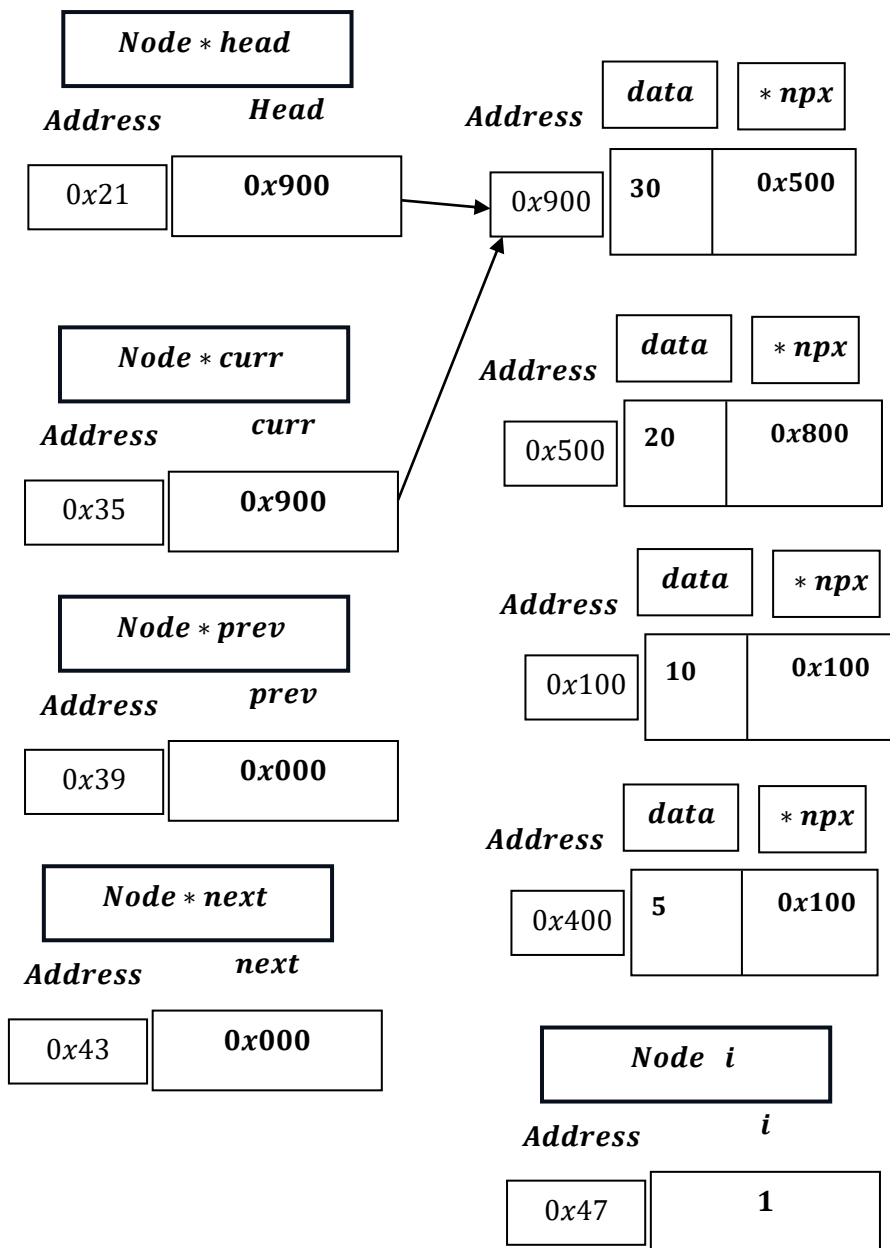
*if pos is equal to 1 , it will run: insertAtBeginning(data);  
and return void and exit from the function.*

---

*If  $pos > 1$ , then we have 4 nodes, say:*



```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;  
int i = 1;
```



```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

***let , pos = 4***

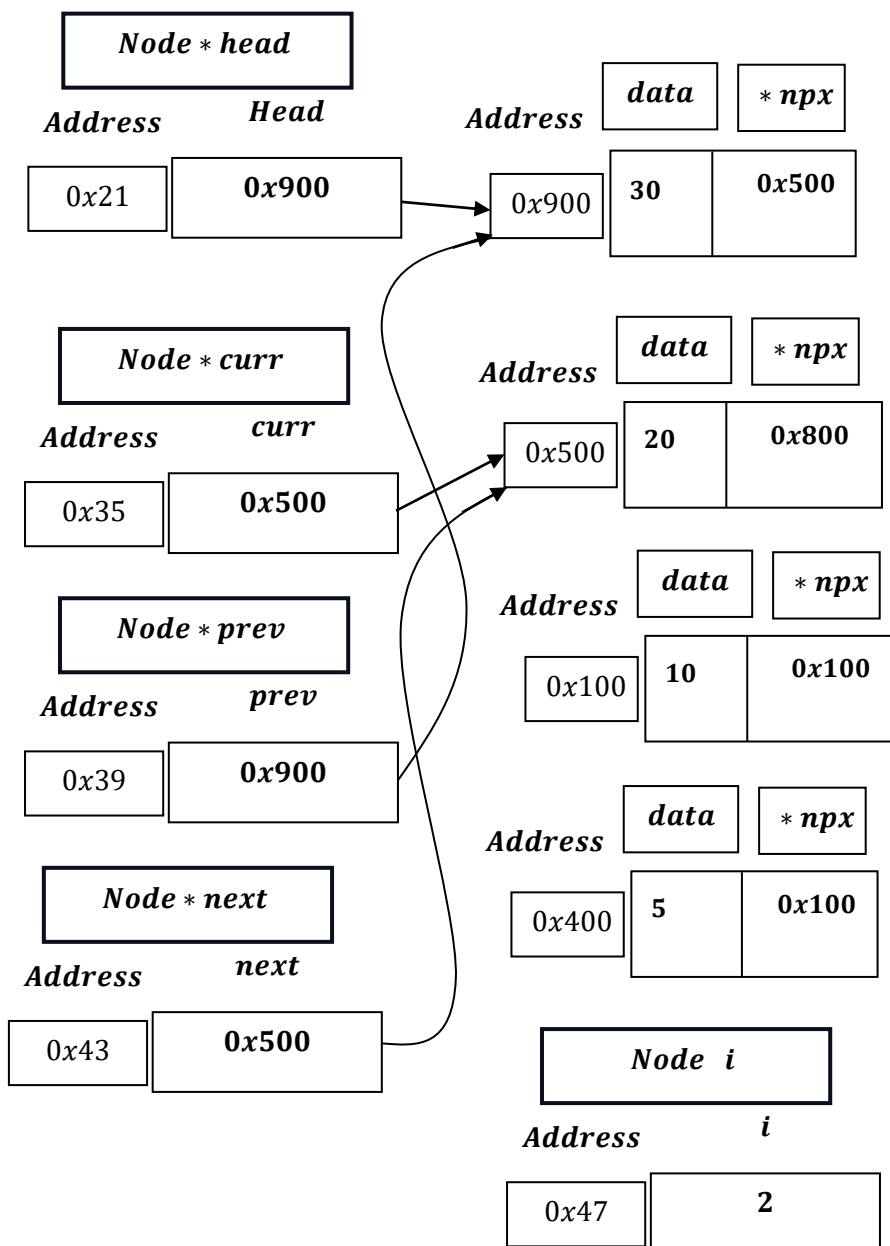
***curr(0x900) ≠ 0x000 And i < (4 − 1 = 3) ⇒ 1 < 3 is true:***

***next = XOR(0x000, curr → npx ⇒ 0x500) = 0x500.***

***prev = curr = 0x900;***

***curr = next = 0x500;***

***i = i + 1 = 1 + 1 = 2;***



$curr(0x500) \neq 0x000$  And  $i < (4 - 1 = 3) \Rightarrow 2 < 3$  is true:

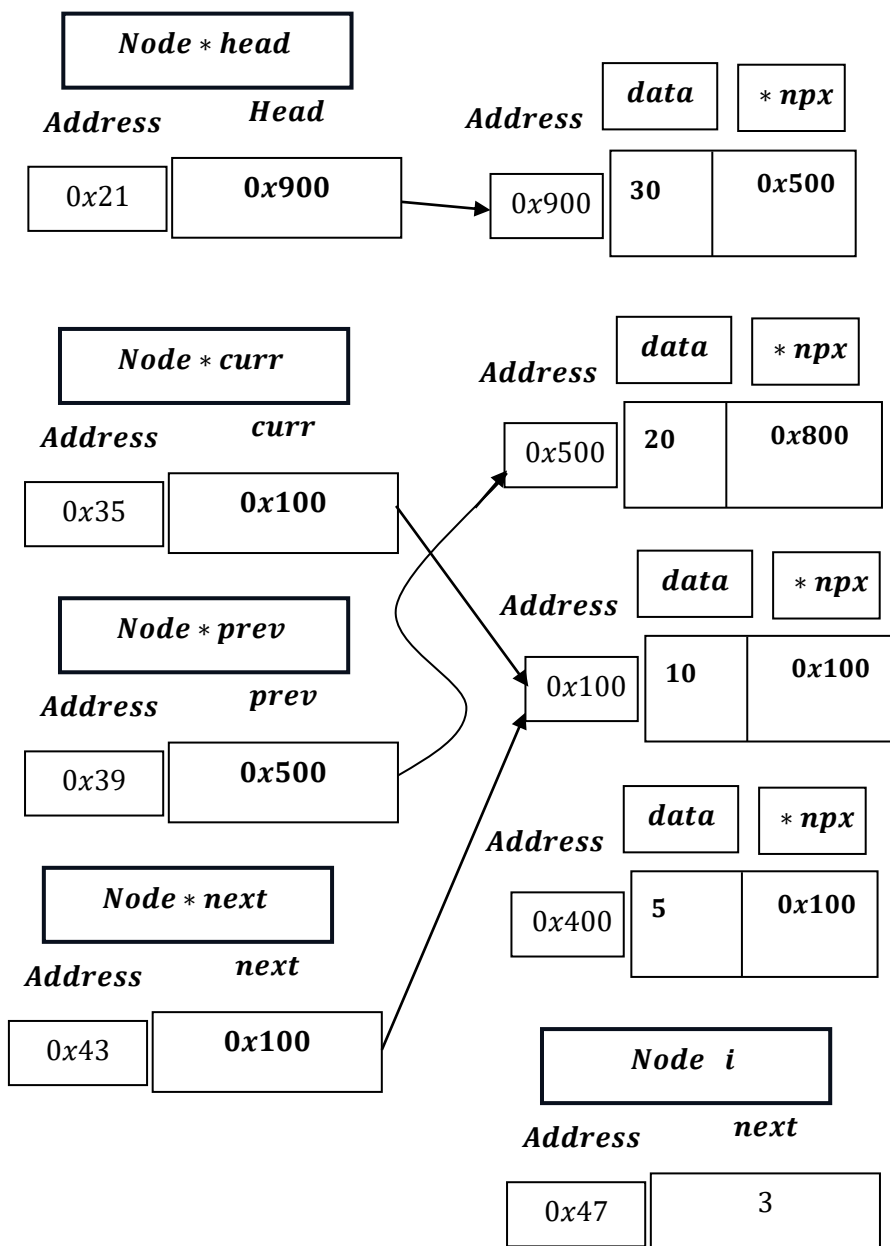
$next = XOR(0x900, curr \rightarrow npx \Rightarrow 0x800)$

$= 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500;$

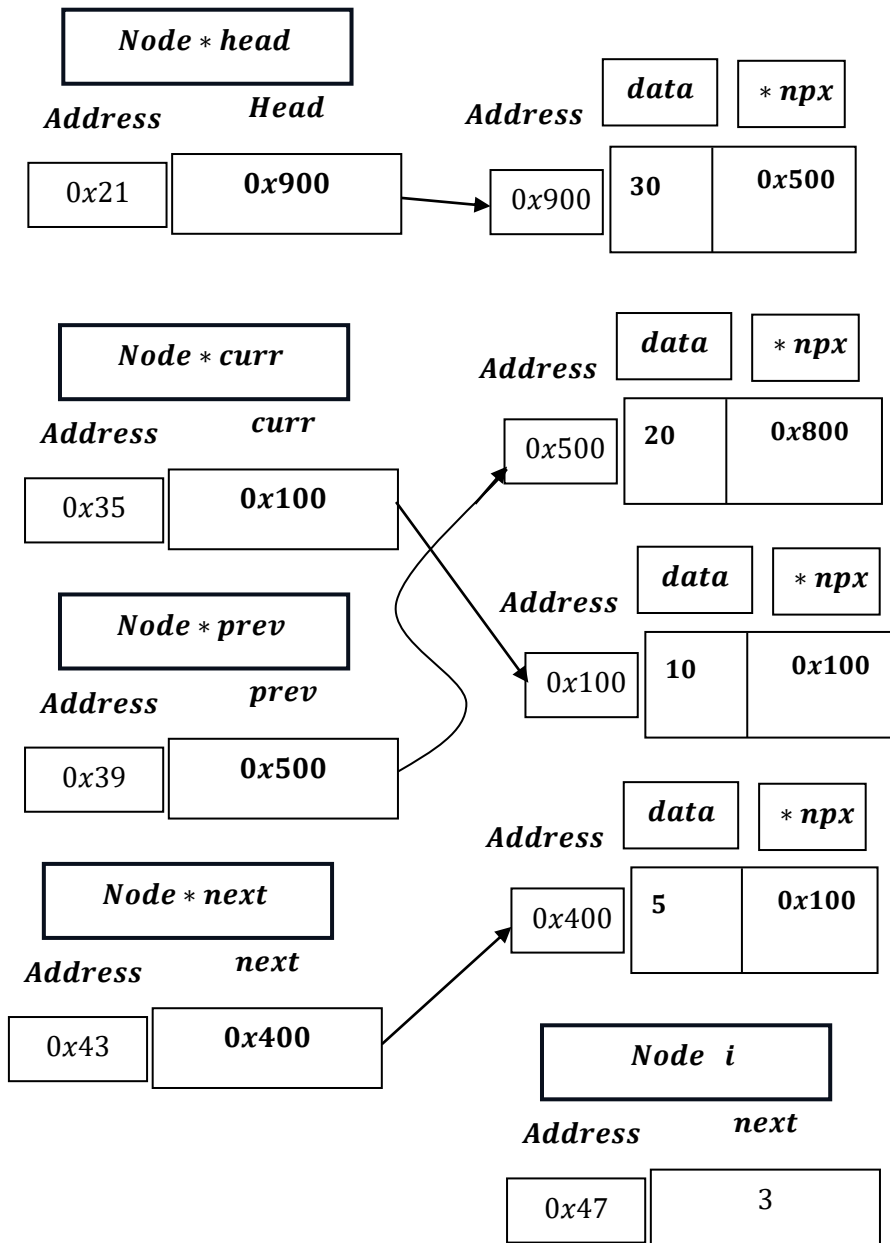
$curr = next = 0x100;$

$i = i + 1 = 2 + 1 = 3;$



$curr(0x100) \neq 0x000$  And  $i < (4 - 1 = 3) \Rightarrow 3 < 3$  is false. Hence , loop exits.

```
next = XOR(prev, curr->npx);
next = XOR(0x500, 0x100) = 01012  $\oplus$  00012 = 01002 = 410 = 0x400
```



```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

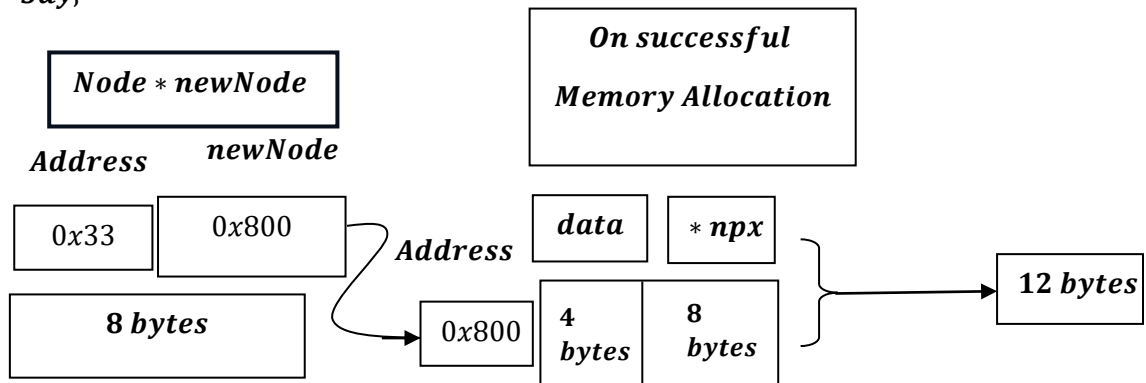
→ *Therefore ,the size of the Node structure is typically 12 bytes( 4 bytes for data + 8 bytes for npx) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

*What does it mean?*

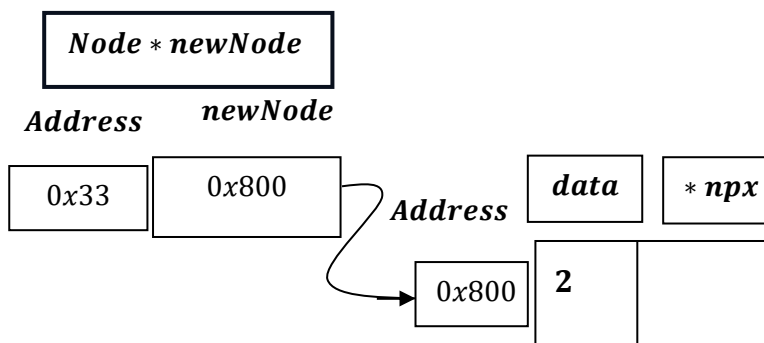
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier ,the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



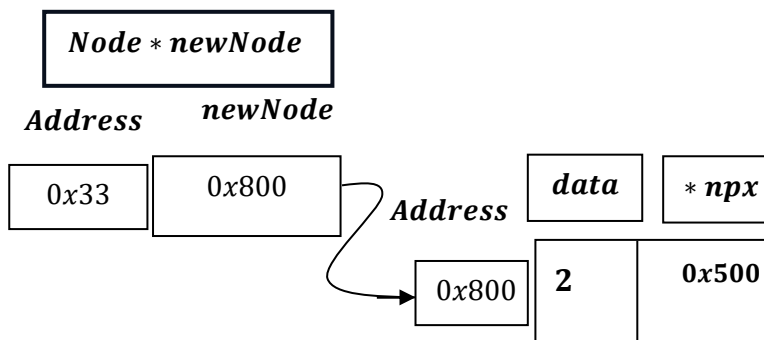
```
newNode->data = data;
```

Say,  $data = 2$ . Then,  $newNode \rightarrow data = data = 2$ .

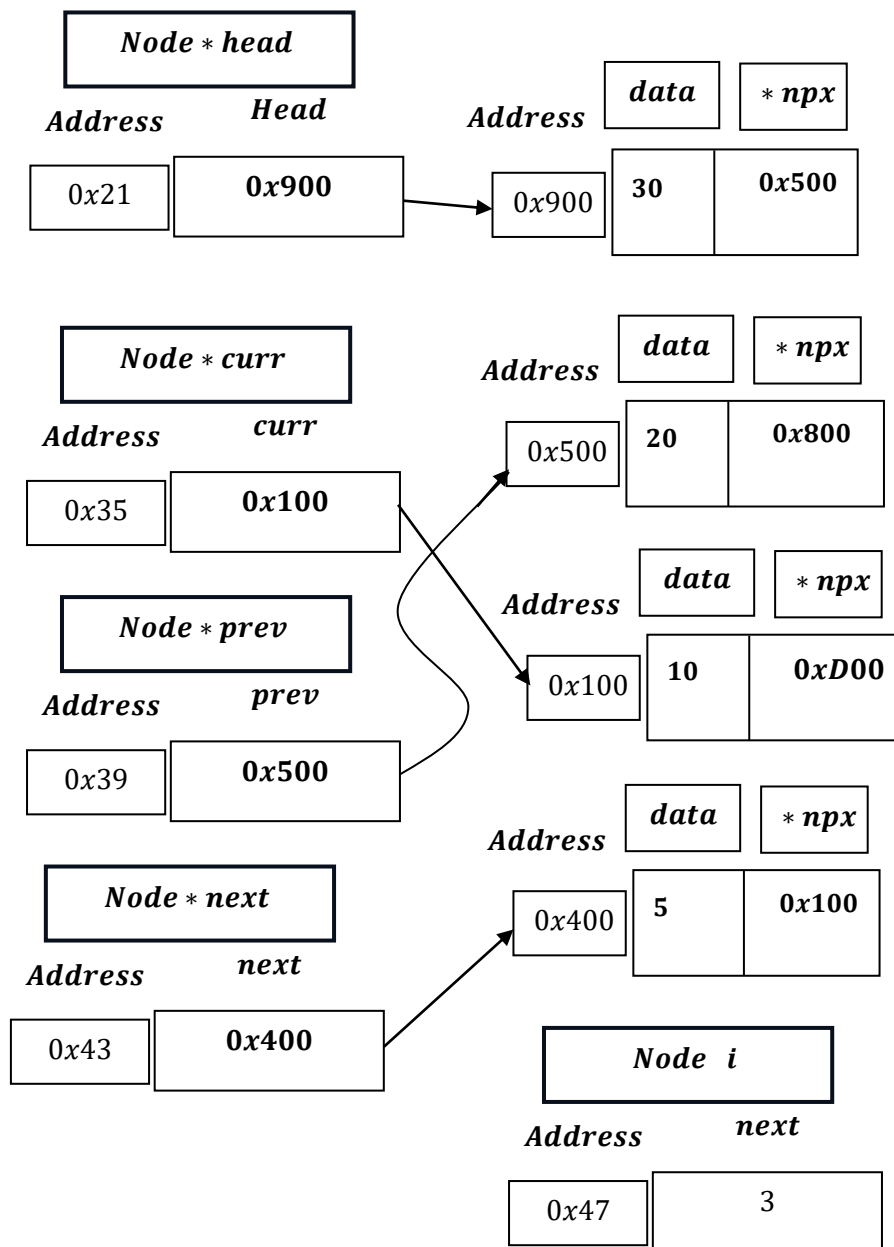


```
newNode->npx = XOR(curr, next);
```

Say,  $newNode \rightarrow npx = XOR(0x100, 0x400) = 0001_2 \oplus 0100_2 = 0101_2 = 0x500$



```
curr->npx = XOR(prev, newNode);
```

$$curr \rightarrow npx = XOR(0x500, 0x800) = 0101_2 \oplus 1000_2 = 1101_2 = 13_{10} = 0xD00$$


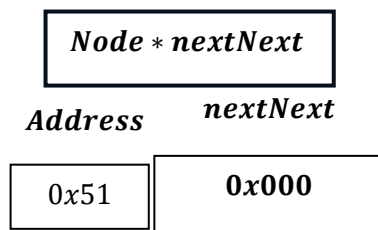
```

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}

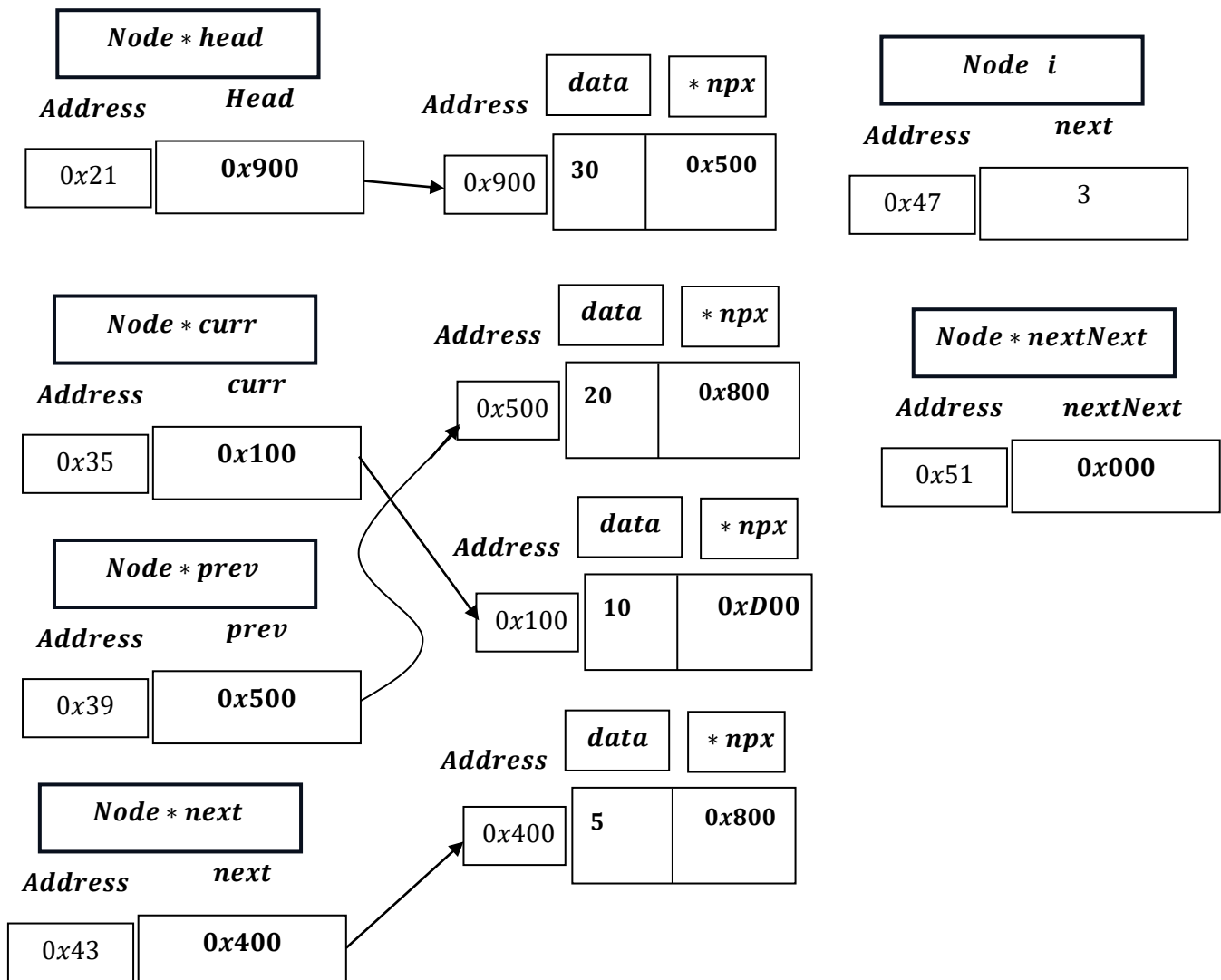
```

*if next  $\neq$  nullptr is true:*

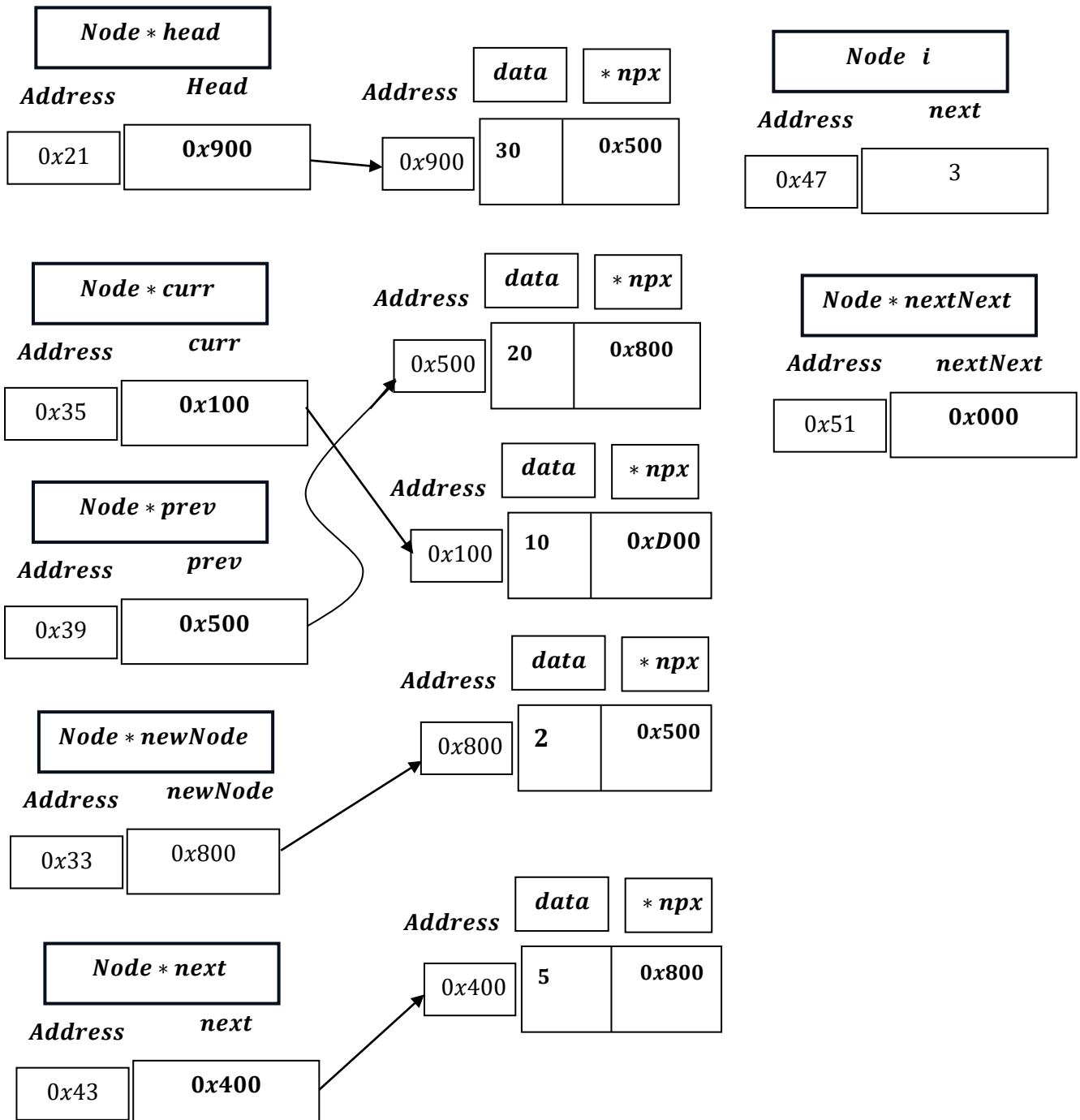
$$\text{Node} * \text{nextNext} = \text{XOR}(\text{curr}, \text{next} \rightarrow \text{npx}) = 0x100 \oplus 0x100 = 0x000$$



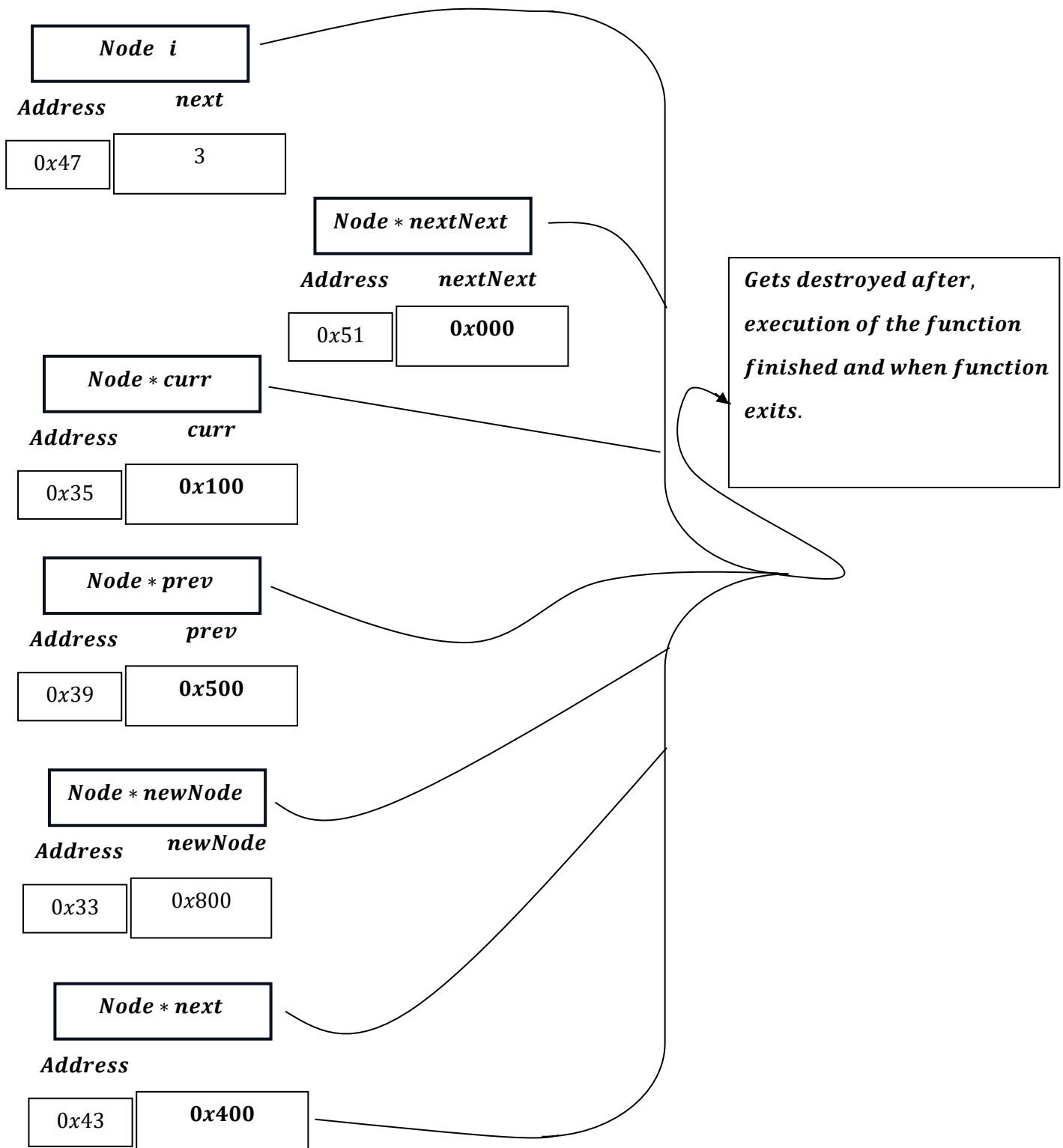
$$\text{next} \rightarrow \text{npx} = \text{XOR}(\text{newNode}, \text{nextNext}) = 0x800 \oplus 0x000 = 0x800(\text{Direct})$$



*Now, we have:*



*As , functions ends:*



*And we are left with:*

